ARM + FPGA 通信案例开发手册

Revision History

Draft Date	Revision No.	Description
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 3
1　PCIe 通信案例 4
1.1　简介 4
1.2　案例说明 5
1.3　案例测试 6
1.4　案例编译（ARM 端） 13
1.5　关键代码说明（ARM 端） 15
1.6　基于 SDMA 的 DDR 读写测试 17
更多帮助 22

前 言

本文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu 20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71

Linux SDK：Real-Time Edge Software 2.5

进行本文档操作前，请先按照《调试工具安装》文档安装 USB 转串口驱动、SecureCRT 串口调试终端等相关软件，并使用 Type-C 线将评估板 USB TO UART2 调试串口连接至 PC 机 USB 接口。

本文主要介绍 NXP i.MX 8M Plus 与 FPGA 的通信案例。ARM 端案例位于产品资料“4-软件资料\Demo\platform-demos\dma_memcpy_demo\”目录下，FPGA 端案例位于产品资料“4-软件资料\Demo\FPGA-demos\bram_pcie\”目录下，具体说明如下：

表 1

<table>
<tr><th colspan="3">目录</th><th>说明</th></tr>
<tr><td rowspan="6">dma_memcpy_demo</td><td colspan="2">bin</td><td>ARM 端可执行程序</td></tr>
<tr><td colspan="2">driver</td><td>驱动镜像和源码</td></tr>
<tr><td rowspan="2">dts</td><td>bin</td><td>设备树镜像</td></tr>
<tr><td>src</td><td>设备树源码</td></tr>
<tr><td colspan="2">src</td><td>应用程序源码</td></tr>
<tr><td colspan="2">doc</td><td>官方数据手册</td></tr>
<tr><td>bram_pcie</td><td colspan="2">bin</td><td>FPGA 端可执行程序</td></tr>
</table>

1 PCIe 通信案例

1.1 简介

在 i.MX 8M Plus 处理器的 PCIe 控制器内嵌了一个多通道 DMA 控制器，用于支持 DMA 传输。由于 PCIe DMA 并不支持内核提供的 DMA 驱动框架，因此需要编写相关驱动初始化此 DMA 控制器。

本案例演示使用 PCIe DMA 进行读写 FPGA BRAM，PCIe DMA 读写访问框图如下所示。

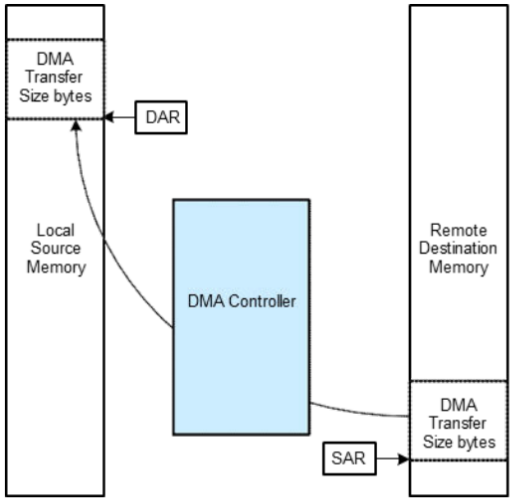

Figure 11-49. Read Transfer: SAR and DAR for Read Channel 0

图 1

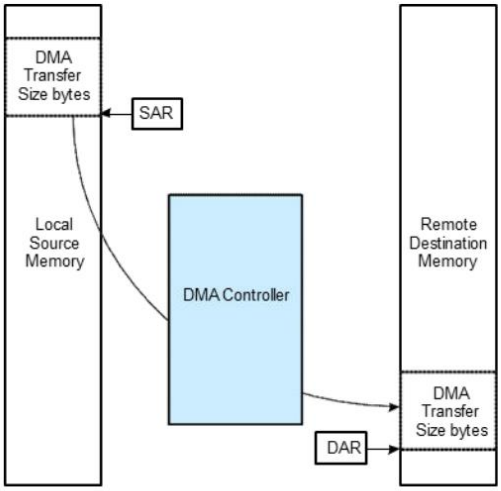

Figure 11-48. Write Transfer: SAR and DAR for Write Channel 0

图 2

关于 SDMA 和 PCIe DMA 详细内容请参考“4-软件资料\Demo\platform-demos\dma_memcpy_demo\doc\”目录下《i.MX 8M Plus Applications Processor》文档。

1.2 案例说明

案例功能：演示基于 PCIe 总线对 FPGA BRAM 进行读写测试。应用程序通过 ioctl 函数发送命令开启 DMA 传输数据后，等待驱动上报 input 事件；当应用层接收到 input 事件，说明 DMA 传输数据完成。

使用 PCIe DMA 读写 FPGA BRAM 程序流程如下图所示。

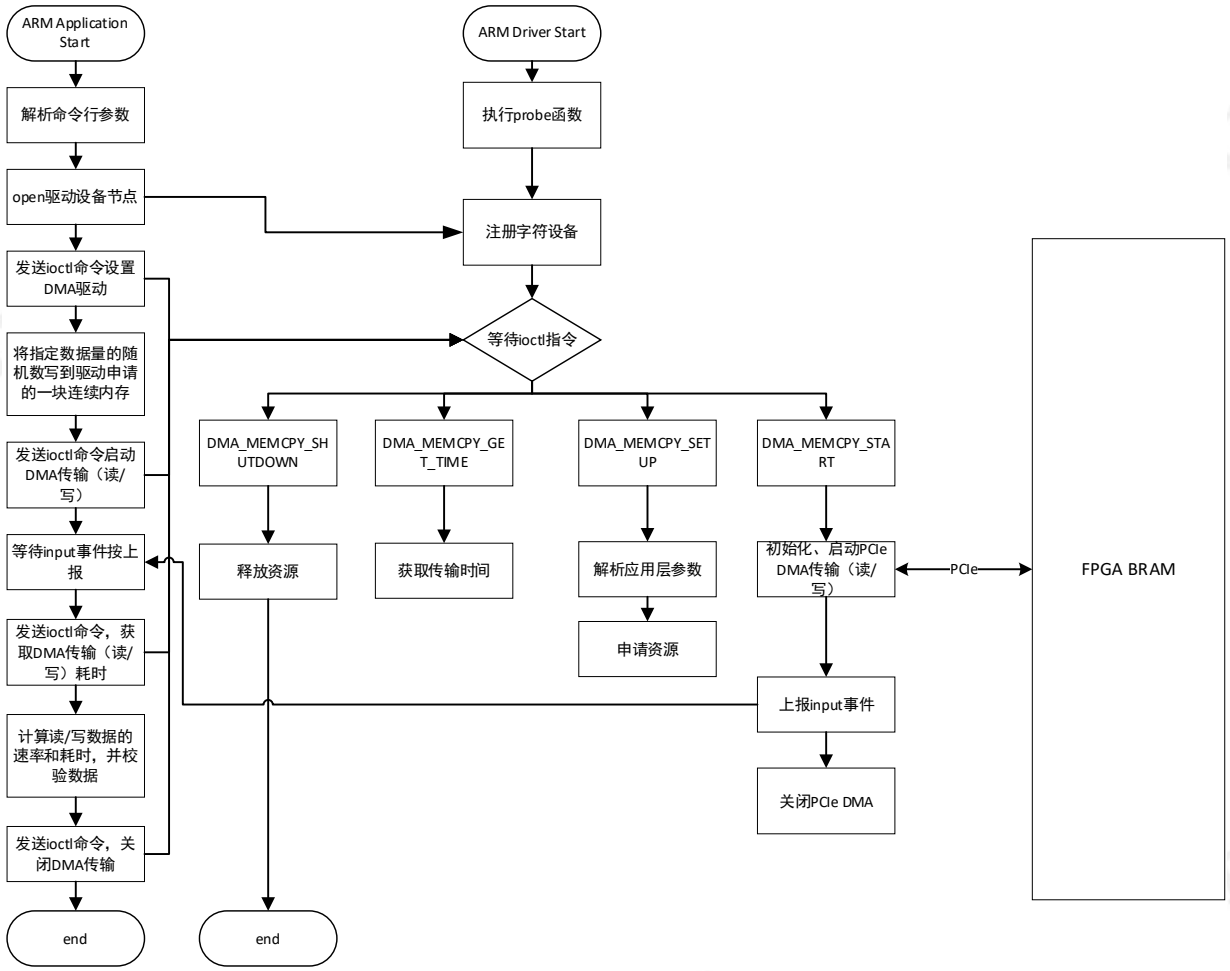

图 3

程序原理说明：

ARM 端：

a) 采用 DMA 方式；

b) 将数据写至 dma_memcpy 驱动申请的连续内存空间（位于 DDR）；

c) 配置 DMA，如源地址、目标地址、传输的数据大小等；

d) 写操作：通过 ioctl 函数启动 DMA，通过 PCIe 总线将数据搬运至 FPGA BRAM；

e) 程序接收驱动上报 input 事件后，将通过 ioctl 函数获取 DMA 搬运数据耗时，并计算 DMA 传输速率（即写速率）；

f) 读操作：通过 ioctl 函数启动 DMA，通过 PCIe 总线将 FPGA BRAM 中的数据搬运至 dma_memcpy 驱动申请的连续内存空间（位于 DDR）；

g) 程序接收驱动上报 input 事件后，将数据从内核空间读取至用户空间，然后校验数据，同时通过 ioctl 函数获取 DMA 搬运数据耗时，并计算 DMA 传输速率（即读速率）。

FPGA 端：

a) 实现 PCIe Endpoint 功能；

b) 处理 PCIe RC 端发起的 PCIe BAR0 空间读写事务；

c) 将 PCIe BAR0 读写数据缓存至 FPGA BRAM 中。

1.3 案例测试

请使用 M.2 Key M 转 PCI-E 4X 转接板（品牌：华高智，型号：PCI-E 4X TO M.2 M）插至 M.2 MKey 转 B + M Key 转接板，然后将 M.2 MKey 转 B + M Key 转接板插至评估板 M.2 PCIe NVMe/5G 插槽(CON20)，将评估板 M.2 PCIe NVMe 接口与创龙科技 TLA7-EVM 评估板 PCIe 接口连接，并将 TLA7-EVM 评估板通过创龙科技 TL-DLC10 下载器连接至 PC 机，硬件连接如下图所示。

图 4 M.2 Key M 转 PCI-E 4X 转接板

图 5 M.2 MKey 转 B + M Key 转接板

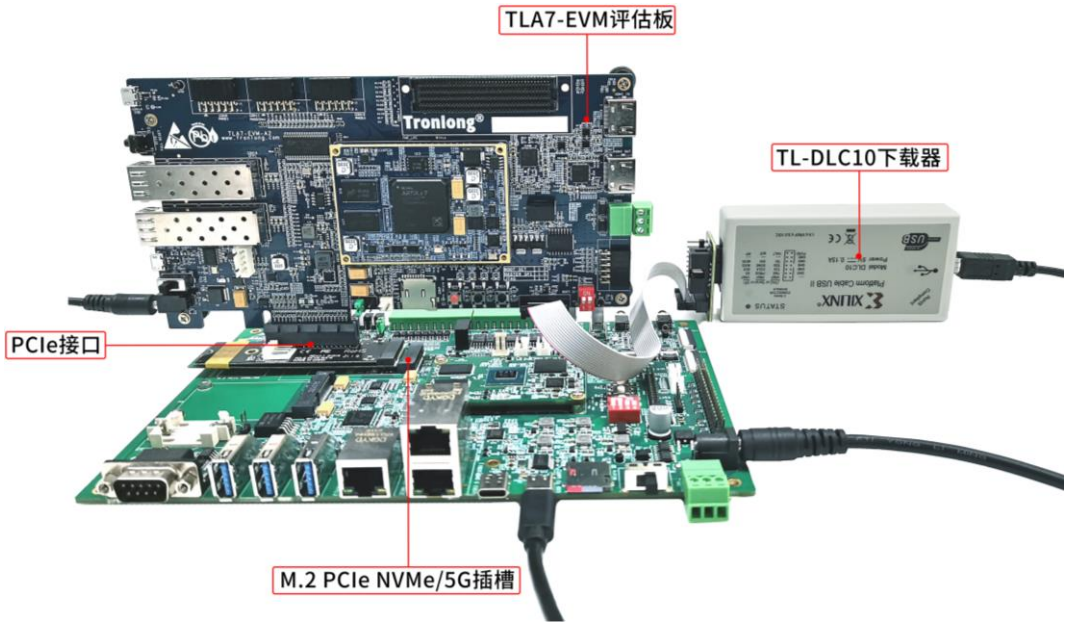

图 6

请将创龙科技 TLA7-EVM 评估板上电启动，加载运行 FPGA 案例"bram_pcie\bin\"目录下的可执行程序。该程序主要实现 PCIe Endpoint 功能，采用 PCIe 同源时钟模式。

请将案例"dma_memcpy_demo\dts\bin\"目录下的 tlimx8mp-evm-reserved-memory.dtb 设备树文件拷贝至评估板文件系统。

备注：tlimx8mp-evm-reserved-memory.dtb 设备树文件主要配置预留 DDR 内存用于读写 DDR 测试、匹配设备驱动。

```
root@IMX8MP-Tronlong:~# ls
tlimx8mp-evm-reserved-memory.dtb
root@IMX8MP-Tronlong:~#
```

图 7

执行如下命令，备份默认设备树文件，替换设备树文件至 Linux 系统启动卡，评估板重启生效。

Target# cd /run/media/mmcblk1p1/

Target# cp tlimx8mp-evm.dtb tlimx8mp-evm-bak.dtb

Target# cp ~/tlimx8mp-evm-reserved-memory.dtb tlimx8mp-evm.dtb

Target# sync

Target# reboot

```
root@IMX8MP-Tronlong:~# cd /run/media/mmcblk1p1/
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# cp tlimx8mp-evm.dtb tlimx8mp-evm-bak.dtb
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# cp ~/tlimx8mp-evm-reserved-memory.dtb tlimx8mp-evm.dtb
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# sync
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# reboot
```

图 8

请将案例 bin 目录下的可执行程序 dma_memcpy_demo 和"driver\bin\"目录下的驱动程序 pcie_dma_memcpy.ko 拷贝至文件系统任意目录下。

```
root@IMX8MP-Tronlong:~# ls
dma_memcpy_demo  pcie_dma_memcpy.ko  tlimx8mp-evm-reserved-memory.dtb
root@IMX8MP-Tronlong:~#
```

图 9

进入评估板文件系统，在可执行程序所在目录执行如下命令查看程序参数。

Target# ./dma_memcpy_demo -h

参数解析：

-a：设置 DDR/PCIe 地址；

-s：设置传输数据大小（单位：Byte）；

-c：设置循环读写次数；

-d：设置输入设备；

-v：显示版本信息；

-h：显示帮助内容。

```
root@IMX8MP-Tronlong:~# ./dma_memcpy_demo -h
Usage: dma_memcpy_demo [options]

Options:
 -a | --address        DDR/PCIe address
 -s | --size           Data size (Unit: Byte)
 -c | --cycles         Loop read and write times
 -d | --device         Input device
 -v | --version        Display version information
 -h | --help           Show help content

Example:
  # ./dma_memcpy_demo -a 0x01000000 -s 2048 -c 10000 -d /dev/input/event2
root@IMX8MP-Tronlong:~#
```

图 10

执行如下命令，调整打印级别，避免调试串口终端打印内核信息影响观察测试结果。

Target# echo 1 4 1 7 > /proc/sys/kernel/printk

```
root@IMX8MP-Tronlong:~# echo 1  4  1  7 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~#
```

图 11

执行如下命令，查看评估板是否识别成功 PCIe 设备。

备注：若无法正常识别 PCIe 设备，请使评估板断电后，重新拔插 PCIe 设备，再重新启动评估板。

Target# lspci

```
root@IMX8MP-Tronlong:~# lspci
00:00.0 PCI bridge: Synopsys, Inc. DWC_usb3 / PCIe bridge (rev 01)
01:00.0 Memory controller: Xilinx Corporation Device 7022
root@IMX8MP-Tronlong:~#
```

图 12

执行如下命令，使能 PCIe 设备。

Target# echo 1 > /sys/bus/pci/devices/0000:01:00.0/enable

```
root@IMX8MP-Tronlong:~# echo 1 > /sys/bus/pci/devices/0000:01:00.0/enable
root@IMX8MP-Tronlong:~#
```

图 13

执行如下命令，查看 PCIe BAR 空间地址。从下图可见，PCIe BAR 空间映射至 0x181 00000 地址，大小为 64KByte。

Target# lspci -vvv

```
root@IMX8MP-Tronlong:~# lspci -vvv
00:00.0 PCI bridge: Synopsys, Inc. DWC_usb3 / PCIe bridge (rev 01) (prog-if 00 [
Normal decode])
        Control: I/O+ Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr- Step
ping- SERR+ FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort-
<MAbort- >SERR- <PERR- INTx-
        Latency: 0
        Interrupt: pin A routed to IRQ 250
        Region 0: Memory at 18000000 (32-bit, non-prefetchable) [size=1M]
        Bus: primary=00, secondary=01, subordinate=ff, sec-latency=0
        I/O behind bridge: 0000f000-00000fff [disabled]
        Memory behind bridge: 18100000-181fffff [size=1M]
        Prefetchable memory behind bridge: fff00000-000fffff [disabled]
        Secondary status: 66MHz- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort-
<MAbort- <SERR- <PERR-
        Expansion ROM at 18200000 [virtual] [disabled] [size=64K]
        BridgeCtl: Parity- SERR+ NoISA- VGA- VGA16- MAbort- >Reset- FastB2B-
                PriDiscTmr- SecDiscTmr- DiscTmrStat- DiscTmrSERREn-
```

图 14

```
01:00.0 Memory controller: Xilinx Corporation Device 7022
        Subsystem: Xilinx Corporation Device 0007
        Control: I/O- Mem+ BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Step
ping- SERR- FastB2B- DisINTx-
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort-
<MAbort- >SERR- <PERR- INTx-
        Region 0: Memory at 18100000 (32-bit, non-prefetchable) [size=64K]
        Capabilities: [40] Power Management version 3
                Flags: PMEClk- DSI+ D1- D2- AuxCurrent=0mA PME(D0-,D1-,D2-,D3hot
-,D3cold-)
                Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
        Capabilities: [48] MSI: Enable- Count=1/1 Maskable- 64bit+
                Address: 0000000000000000  Data: 0000
        Capabilities: [60] Express (v2) Endpoint, MSI 00
                DevCap: MaxPayload 256 bytes, PhantFunc 1, Latency L0s <64ns, L1
 <1us
                        ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowe
rLimit 0.000W
                DevCtl: CorrErr- NonFatalErr- FatalErr- UnsupReq-
                        RlxOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop+
                        MaxPayload 128 bytes, MaxReadReq 512 bytes
                DevSta: CorrErr- NonFatalErr- FatalErr- UnsupReq- AuxPwr- TransP
end-
                LnkCap: Port #0, Speed 5GT/s, Width x2, ASPM L0s, Exit Latency L
0s unlimited
                        ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp-
                LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk+
                        ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
                LnkSta: Speed 5GT/s (ok), Width x1 (downgraded)
                        TrErr- Train- SlotClk+ DLActive- BWMgmt- ABWMgmt-
                DevCap2: Completion Timeout: Not Supported, TimeoutDis- NROPrPrP
- LTR-
                         10BitTagComp- 10BitTagReq- OBFF Not Supported, ExtFmt-
EETLPPrefix-
                         EmergencyPowerReduction Not Supported, EmergencyPowerRe
ductionInit-
                         FRS- TPHComp- ExtTPHComp-
                         AtomicOpsCap: 32bit- 64bit- 128bitCAS-
                DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR- OBFF
 Disabled,
                         AtomicOpsCtl: ReqEn-
                LnkCtl2: Target Link Speed: 5GT/s, EnterCompliance- SpeedDis-
                         Transmit Margin: Normal Operating Range, EnterModifiedC
ompliance- ComplianceSOS-
                         Compliance De-emphasis: -6dB
                LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete-
EqualizationPhase1-
                         EqualizationPhase2- EqualizationPhase3- LinkEqualizatio
nRequest-
                         Retimer- 2Retimers- CrosslinkRes: unsupported
        Capabilities: [100 v1] Device Serial Number 00-00-00-00-00-00-00-00

root@IMX8MP-Tronlong:~#
```

图 15

进入评估板文件系统执行如下命令，加载用于对 FPGA BRAM 读写测试的 pcie_dma_memcpy.ko 驱动程序。

Target# insmod -f pcie_dma_memcpy.ko

```
root@IMX8MP-Tronlong:~# insmod -f pcie_dma_memcpy.ko
root@IMX8MP-Tronlong:~#
```

图 16

执行如下命令，查看事件号。

Target# cat /proc/bus/input/devices

```
root@IMX8MP-Tronlong:~# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="gpio-keys"
P: Phys=gpio-keys/input0
S: Sysfs=/devices/platform/gpio-keys/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=100003
B: KEY=100000 0 0

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="input_dev_demo"
P: Phys=
S: Sysfs=/devices/platform/dma_memory/input/input1
U: Uniq=
H: Handlers=event1
B: PROP=0
B: EV=11
B: MSC=8

root@IMX8MP-Tronlong:~#
```

图 17

执行如下命令，将随机数据先写入 FPGA BRAM，再从 FPGA BRAM 读出。测试完成后，程序将会打印最终测试结果，包含读写平均传输耗时、读写平均传输速率、读写错误统计等信息。

Target# ./dma_memcpy_demo -a 0x18100000 -s 65536 -c 1000 -d /dev/input/event1

```
root@IMX8MP-Tronlong:~# ./dma_memcpy_demo -a 0x18100000 -s 65536 -c 1000 -d /dev/input/event1
Open dma & input device node...
DMA: write: (time: 156 us, rate: 399.72 MB/s, app time: 50 us, app rate: 1236.65 MB/s, write_error: 0)
read: (time: 170 us, rate: 367.64 MB/s, app time: 205 us, app rate: 305.15 MB/s, read_error: 0)

root@IMX8MP-Tronlong:~#
```

图 18

表 2

write/read	写操作	读操作
time	DMA 传输耗时，本次测试为 156us	DMA 传输耗时，本次测试为 170us
rate	DMA 传输速率，本次测试为 399.72MB/s	DMA 传输速率，本次测试为 367.64MB/s
app time	应用层写数据至驱动 Buffer 耗时，本次测试为 50us	应用层从驱动 Buffer 读取数据耗时，本次测试为 205us
app rate	应用层写数据至驱动 Buffer 速率，本次测试为 1236.65MB/s	应用层从驱动 Buffer 读取数据速率，本次测试为 305.15MB/s
write_error/read_error	写数据过程中出错次数，本次测试为 0	读数据过程中出错次数，本次测试为 0

PCIe Gen2 编码开销为 8b/10b，PCIe Gen2 传输数据的开销如下：

（1） 物理层：Start(1Byte)、End(1Byte)

（2） 数据链路层：Sequence(2Byte)、LCRC(4Byte)

（3） 传输层：Header(12Byte 或 16Byte)、ECRC(4Byte)

PCIe 理论带宽计算如下：

写速率：rate = 5Gb/s * num-lanes * 编码开销 * (MPS/ (MPS + 传输数据的开销))

读速率：rate = 5Gb/s * num-lanes * 编码开销 * (RCB/ (RCB + 传输数据的开销))

执行"lspci -vvv"命令可查询到：

num-lanes 为 1，MaxPayload 为 128Byte，RCB 为 64Byte

为了计算协议消耗更高的理论带宽，取 TLP header size =16Byte，因此可计算到 PCIe Gen2 传输数据的开销为 28 字节。可计算得到理论速率为：

写速率：rate = 5Gb/s * 1 * 8/10 * (128 / (128 + 28)) ≈ 420.10MB/s

读速率：rate = 5Gb/s * 1 * 8/10 * (64 / (64 + 28)) ≈ 356.17MB/s

测试完成后，如需恢复默认的设备树文件，请执行如下命令，将上述备份的设备树文件恢复，然后评估板重启生效。

Target# cd /run/media/mmcblk1p1/

Target# cp tlimx8mp-evm-bak.dtb tlimx8mp-evm.dtb

Target# sync

Target# reboot

```
root@IMX8MP-Tronlong:~# cd /run/media/mmcblk1p1/
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# cp tlimx8mp-evm-bak.dtb tlimx8mp-evm.dtb
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# sync
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# reboot
```

图 19

1.4 案例编译（ARM 端）

请将产品资料“4-软件资料\Demo\platform-demos\dma_memcpy_demo\”案例源码拷贝至 Ubuntu 工作目录。

（1） 应用程序编译

进入案例应用程序源码目录，执行如下命令，配置应用程序交叉编译工具链环境变量，然后执行 make 命令进行编译，编译完成将会在当前目录下生成 dma_memcpy_demo 可执行程序。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/Demo/platform-demos/dma_memcpy_demo/src$ ls
dma_memcpy_demo.c  Makefile
```

图 20

```
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/src$ source /home/tronlong/SDK/envi
ronment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall
 -O2    -c -o dma_memcpy_demo.o dma_memcpy_demo.c
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORT
IFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlon
g/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall
 -O2  -o dma_memcpy_demo dma_memcpy_demo.o
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/src$ ls
dma_memcpy_demo  dma_memcpy_demo.c  dma_memcpy_demo.o  Makefile
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/src$
```

图 21

（2） 驱动程序编译

进入案例驱动程序源码目录，执行如下命令进行编译，编译完成将会在当前目录下生成 pcie_dma_memcpy.ko、sdma_memcpy.ko 驱动程序。

备注：sdma_memcpy.ko 驱动程序用于读写 DDR，pcie_dma_memcpy.ko 驱动程序用于读写 FPGA BRAM。

Host# make KDIR=/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/

```
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/driver/src$ ls
Makefile  pcie_dma_memcpy.c  sdma_memcpy.c
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/driver/src$ make KDIR=/home/tronlon
g/IMX8MP/Kernel/Linux-5.15.71-rt51/
make -C /home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51/ M=/home/tronlong/dma_memcpy_de
mo/driver/src modules ARCH=arm64 CROSS_COMPILE=aarch64-poky-linux-
make[1]: Entering directory '/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51'
  CC [M]  /home/tronlong/dma_memcpy_demo/driver/src/sdma_memcpy.o
  CC [M]  /home/tronlong/dma_memcpy_demo/driver/src/pcie_dma_memcpy.o
  MODPOST /home/tronlong/dma_memcpy_demo/driver/src/Module.symvers
  CC [M]  /home/tronlong/dma_memcpy_demo/driver/src/pcie_dma_memcpy.mod.o
  LD [M]  /home/tronlong/dma_memcpy_demo/driver/src/pcie_dma_memcpy.ko
  CC [M]  /home/tronlong/dma_memcpy_demo/driver/src/sdma_memcpy.mod.o
  LD [M]  /home/tronlong/dma_memcpy_demo/driver/src/sdma_memcpy.ko
make[1]: Leaving directory '/home/tronlong/IMX8MP/Kernel/Linux-5.15.71-rt51'
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/driver/src$ ls
Makefile            pcie_dma_memcpy.ko     pcie_dma_memcpy.o  sdma_memcpy.mod.c
modules.order       pcie_dma_memcpy.mod    sdma_memcpy.c      sdma_memcpy.mod.o
Module.symvers      pcie_dma_memcpy.mod.c  sdma_memcpy.ko     sdma_memcpy.o
pcie_dma_memcpy.c   pcie_dma_memcpy.mod.o  sdma_memcpy.mod
tronlong@tronlong-virtual-machine:~/dma_memcpy_demo/driver/src$
```

图 22

（3） 设备树编译

请将案例"dts\src\tlimx8mp-evm-reserved-memory.dts"设备树源文件拷贝至内核源码目录"arch/arm64/boot/dts/freescale/"。

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/arch/arm64/boot/dt
s/freescale$ ls tlimx8mp-evm-reserved-memory.dts
tlimx8mp-evm-reserved-memory.dts
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51/arch/arm64/boot/dt
s/freescale$
```

图 23

请在内核源码安装目录下，执行如下命令使能 SDK 环境变量，并在内核源码目录"arch/arm64/boot/dts/freescale/"路径中编译生成 tlimx8mp-evm-reserved-memory.dtb 文件。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make freescale/tlimx8mp-evm-reserved-memory.dtb

```
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ source /home/tron
long/SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ make freescale/tl
imx8mp-evm-reserved-memory.dtb
  DTC     arch/arm64/boot/dts/freescale/tlimx8mp-evm-reserved-memory.dtb
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$ ls arch/arm64/boo
t/dts/freescale/tlimx8mp-evm-reserved-memory.dtb
arch/arm64/boot/dts/freescale/tlimx8mp-evm-reserved-memory.dtb
tronlong@tronlong-virtual-machine:~/IMX8MP/Kernel/Linux-5.15.71-rt51$
```

图 24

1.5 关键代码说明（ARM 端）

（1） 打开设备节点。

```
static int dma_init(struct DMAInfo *dma_info)
{
    if (dma_info == NULL) {
        return -1;
    }

    printf("Open dma & input device node...\n");
    dma_info->dma_fd = open(DMA_MEMCPY_PATH, O_RDWR);
    if(dma_info->dma_fd < 0) {
        printf("can't open %s\n", DMA_MEMCPY_PATH);
        return -1;
    }

    dma_info->input_fd = open(dma_info->params.dev, O_RDONLY | O_NONBLOCK);
    if(dma_info->input_fd < 0) {
        printf("can't open %s\n", dma_info->params.dev);
        return -1;
    }
    printf("Success open dma & input device node.\n\n");

    return 0;
}
```

图 25

（2） 写数据操作。

```
static int dma_write(struct DMAInfo *dma_info)
{
    struct DMASetupParams dma_setup_params;
    unsigned int dev_addr = 0;
    int i = 0, ret = -1;
    struct input_event input_buf;
    int status = 0;
    struct timeval start, end, time_diff;
    fd_set input;
    struct timeval timeout;

    /* Populate the write buffer */
    srand(time(NULL));
    for (i = 0; i < dma_info->params.size / 2; i++) {
        w_buffer[i] = rand() % 0xFFFF;
    }

    /* Setup DMA */
    dma_setup_params.dev_phy_addr           = dma_info->params.address;
    dma_setup_params.transfer_total_size    = dma_info->params.size;
    dma_setup_params.transfer_single_size   = dma_info->params.size;
    /* 0: write, 1: read */
    dma_setup_params.transfer_direction     = 0;
    dev_addr = ioctl(dma_info->dma_fd, DMA_MEMCPY_SETUP, &dma_setup_params);
    if (dev_addr == 0) {
        printf("can't get device addr: 0x%x\n", dev_addr);
        return -1;
    }

    dma_info->vir_addr = mmap(NULL, dma_info->params.size, PROT_READ | PROT_WRITE, MAP_SHARED, dma_info->dma_fd, 0);
    if (dma_info->vir_addr == MAP_FAILED) {
        printf("mmap failed\n");
        return -1;
    }

    /* Write data to the buf belong to dma_memcpy module */
    gettimeofday(&start, NULL);
    neon_copy((unsigned char *)dma_info->vir_addr, (unsigned char *)w_buffer, dma_info->params.size);
    gettimeofday(&end, NULL);

    /* Get application transfer rate and time */
    timersub(&end, &start, &time_diff);
    dma_info->dma_trans_info.app_w_cost_time = time_diff.tv_sec * 1000000.0 + time_diff.tv_usec;
    dma_info->dma_trans_info.app_w_rate = ((dma_info->params.size * 1.0 / (dma_info->dma_trans_info.app_w_cost_time / 1000000.0)) / 1024 / 1024);
```

图 26

```
    /* Start DMA */
    ret = ioctl(dma_info->dma_fd, DMA_MEMCPY_START, 0);
    if (ret != 0) {
        return -1;
    }

    /* Waiting for input event */
    timeout.tv_sec = 1;
    timeout.tv_usec = 0;
    while (!g_quit) {
        FD_ZERO(&input);
        FD_SET(dma_info->input_fd, &input);

        ret = select(dma_info->input_fd + 1, &input, NULL, NULL, &timeout);
        if (ret < 0) {
            status = -1;
            printf("select failed\n");
            goto error;
        } else if (ret == 0) {
            status = -1;
            printf("select timeout\n");
            goto error;
        } else {
            break;
        }
    }

    ret = read(dma_info->input_fd, &input_buf, sizeof(struct input_event));
    if (ret < 0) {
        status = -1;
        goto error;
    }
```

图 27

```
    /* Get the time when DMA transmits data */
    dma_info->dma_trans_info.dma_w_cost_time = ioctl(dma_info->dma_fd, DMA_MEMCPY_GET_TIME, 0);
    dma_info->dma_trans_info.dma_w_rate = ((dma_info->params.size * 1.0 / (dma_info->dma_trans_info.dma_w_cost_time / 1000000.0)) / 1024 / 1024);

error:
    /* Shutdown DMA transfer */
    ret = ioctl(dma_info->dma_fd, DMA_MEMCPY_SHUTDOWN, 0);
    if (ret < 0) {
        status = -1;
    }

    munmap(dma_info->vir_addr, dma_info->params.size);

    return status;
}
```

图 28

（3） 读取数据操作。

```
static int dma_read(struct DMAInfo *dma_info)
{
    struct DMASetupParams dma_setup_params;
    unsigned int dev_addr = 0;
    int status = 0;
    int ret = -1, i = 0;
    struct input_event input_buf;
    struct timeval start, end, time_diff;
    fd_set input;
    struct timeval timeout;

    /* Setup DMA */
    dma_setup_params.dev_phy_addr           = dma_info->params.address;
    dma_setup_params.transfer_total_size    = dma_info->params.size;
    dma_setup_params.transfer_single_size   = dma_info->params.size;
    /* 0: write, 1: read */
    dma_setup_params.transfer_direction     = 1;
    dev_addr = ioctl(dma_info->dma_fd, DMA_MEMCPY_SETUP, &dma_setup_params);
    if (dev_addr == 0) {
        printf("can't get device addr: 0x%x\n", dev_addr);
        return -1;
    }

    /* Start DMA */
    ret = ioctl(dma_info->dma_fd, DMA_MEMCPY_START, 0);
    if (ret != 0) {
        return -1;
    }
```

图 29

```
    /* Waiting for input event */
    timeout.tv_sec = 1;
    timeout.tv_usec = 0;
    while (!g_quit) {
        FD_ZERO(&input);
        FD_SET(dma_info->input_fd, &input);

        ret = select(dma_info->input_fd + 1, &input, NULL, NULL, &timeout);
        if (ret < 0) {
            status = -1;
            printf("select failed\n");
            goto error;
        } else if (ret == 0) {
            status = -1;
            printf("select timeout\n");
            goto error;
        } else {
            break;
        }
    }

    ret = read(dma_info->input_fd, &input_buf, sizeof(struct input_event));
    if (ret < 0) {
        status = -1;
        goto error;
    }
```

图 30

```
    gettimeofday(&start, NULL);
    neon_copy((unsigned char *)r_buffer, (unsigned char *)dma_info->vir_addr, dma_info->params.size);
    gettimeofday(&end, NULL);

    /* Get application transfer rate and time */
    timersub(&end, &start, &time_diff);
    dma_info->dma_trans_info.app_r_cost_time = time_diff.tv_sec * 1000000.0 + time_diff.tv_usec;
    dma_info->dma_trans_info.app_r_rate = ((dma_info->params.size * 1.0 / (dma_info->dma_trans_info.app_r_cost_time / 1000000.0)) / 1024 / 1024);

    /* Verify data */
    for(i = 0; i < dma_info->params.size / 2; i ++) {
        if(r_buffer[i] != w_buffer[i]) {
            dma_info->dma_trans_info.error_count++;
            status = -1;
#ifdef DEBUG
            printf("data verify error at %d, expect val: 0x%x, get value: 0x%x\n", i, w_buffer[i], r_buffer[i]);
#endif
        }
    }

    /* Get the time when DMA transmits data */
    dma_info->dma_trans_info.dma_r_cost_time = ioctl(dma_info->dma_fd, DMA_MEMCPY_GET_TIME, 0);
    dma_info->dma_trans_info.dma_r_rate = ((dma_info->params.size * 1.0 / (dma_info->dma_trans_info.dma_r_cost_time / 1000000.0)) / 1024 / 1024);

error:
    /* Shutdown DMA transfer */
    ret = ioctl(dma_info->dma_fd, DMA_MEMCPY_SHUTDOWN, 0);
    if (ret < 0) {
        status = -1;
    }

    munmap(dma_info->vir_addr, dma_info->params.size);

    return status;
}
```

图 31

1.6 基于 SDMA 的 DDR 读写测试

dma_memcpy_demo 案例同时支持使用 ARM 端 SDMA（Smart Direct Memory Access，智能直接存储区访问）对 DDR 进行读写测试，程序流程如下图所示。

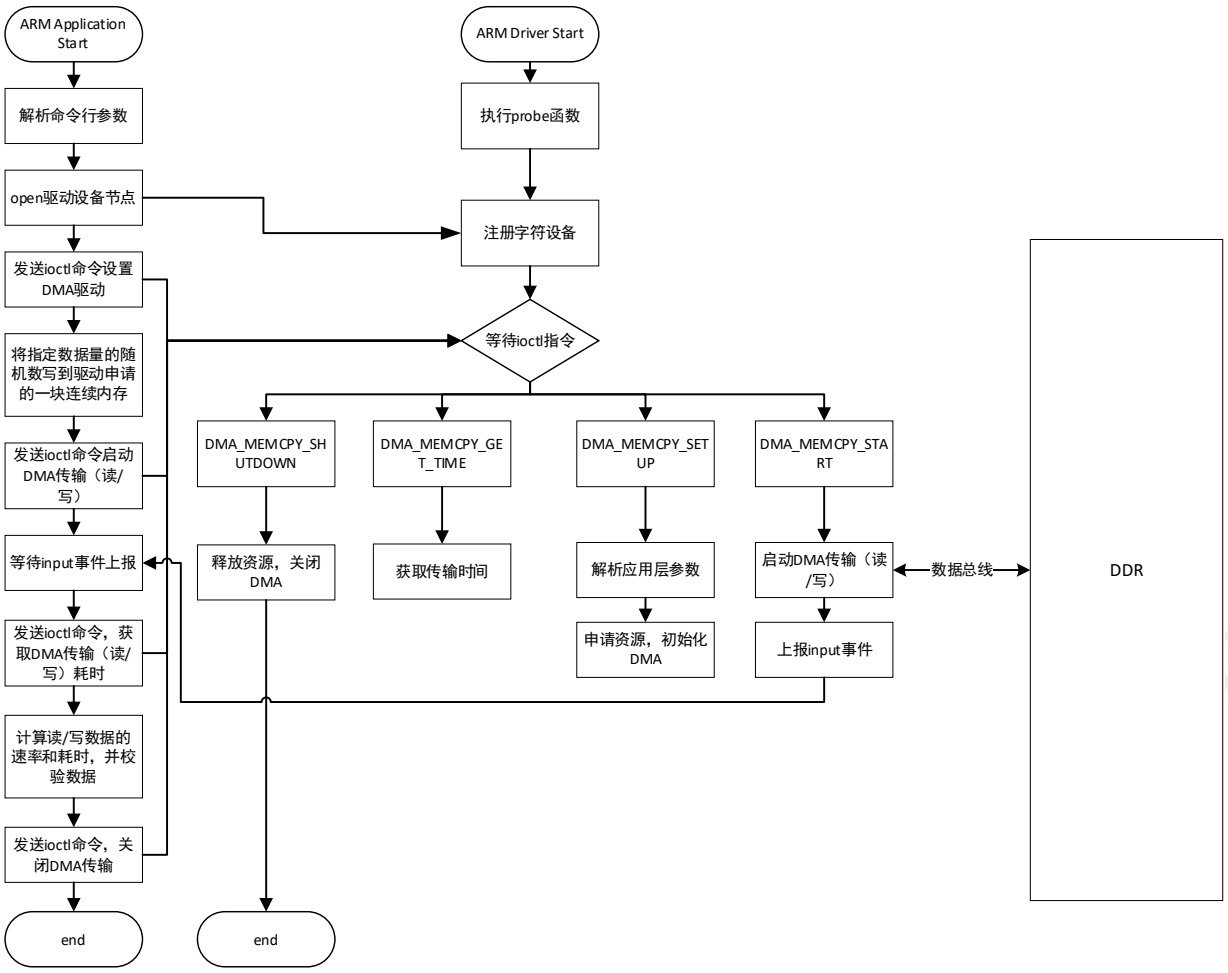

图 32

i.MX 8M Plus 处理器通用的 DMA 称为 SDMA（Smart Direct Memory Access，智能直接存储区访问）。

Linux 内核为通用 DMA 提供了驱动框架，如下图所示。DMA 框架主要分为三个部分：DMA provider、DMA consumer 以及 DMA 核心层。DMA provider，主要包括了各种 SoC 芯片的 DMA 控制器驱动代码，此部分驱动通常由 SoC 芯片厂商编写，实现具体的 DMA 控制器相关初始化。DMA consumer，主要是用于申请 DMA 资源。DMA 核心层，用于提供与硬件无关的接口，对于上层 DMA consumer，它提供了如何请求 DMA 传输的数据结构与接口；对于下层 DMA 控制器驱动，则提供往 Linux 内核注册一个 DMA 设备的接口 API，从而更好地实现 DMA 框架的分层。

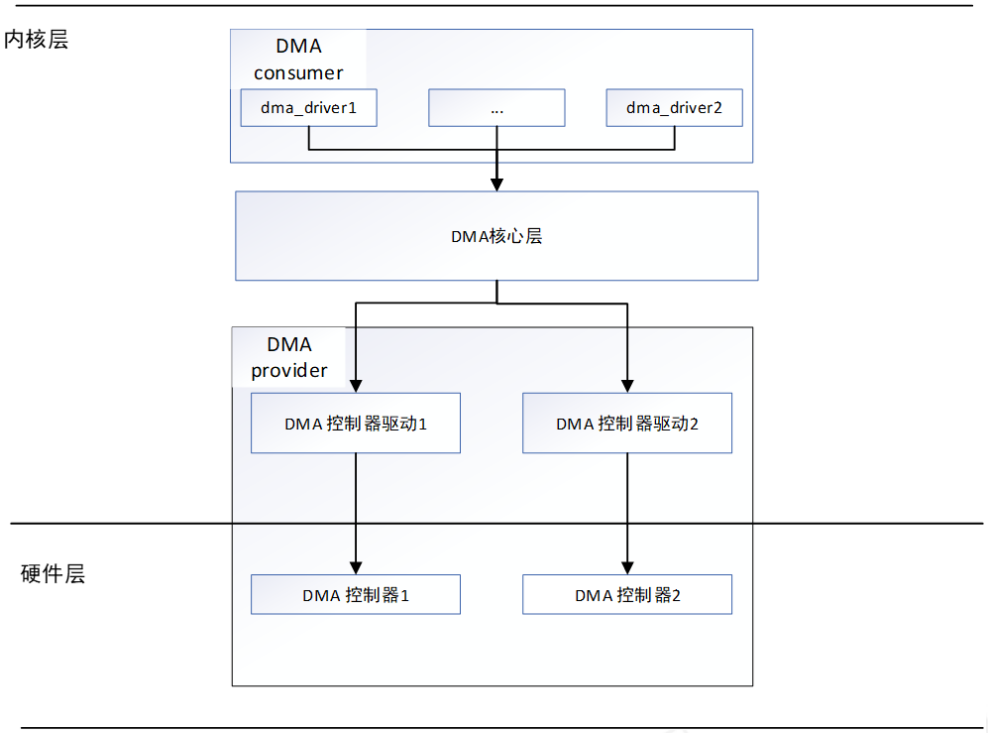

图 33

硬件连接以及替换 tlimx8mp-evm-reserved-memory.dtb 设备树文件等步骤请参考“案例测试”章节。

请将案例 bin 目录下的可执行程序 dma_memcpy_demo 和"driver\bin\"目录下的驱动程序 sdma_memcpy.ko 拷贝至文件系统任意目录下。

```
root@IMX8MP-Tronlong:~# ls
dma_memcpy_demo  pcie_dma_memcpy.ko  sdma_memcpy.ko  tlimx8mp-evm-reserved-memory.dtb
root@IMX8MP-Tronlong:~#
```

图 34

执行如下命令，调整打印级别，避免调试串口终端打印内核信息影响观察测试结果。

Target# echo 1 4 1 7 > /proc/sys/kernel/printk

```
root@IMX8MP-Tronlong:~# echo 1  4  1  7 > /proc/sys/kernel/printk
root@IMX8MP-Tronlong:~#
```

图 35

进入评估板文件系统执行如下命令，加载用于对 DDR 读写测试的 sdma_memcpy.ko 驱动程序。

Target# insmod -f sdma_memcpy.ko

```
root@IMX8MP-Tronlong:~# insmod -f sdma_memcpy.ko
root@IMX8MP-Tronlong:~#
```

图 36

执行如下命令，查看事件号。

Target# cat /proc/bus/input/devices

```
root@IMX8MP-Tronlong:~# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="gpio-keys"
P: Phys=gpio-keys/input0
S: Sysfs=/devices/platform/gpio-keys/input/input0
U: Uniq=
H: Handlers=kbd event0
B: PROP=0
B: EV=100003
B: KEY=100000 0 0

I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="input_dev_demo"
P: Phys=
S: Sysfs=/devices/platform/dma_memory/input/input1
U: Uniq=
H: Handlers=event1
B: PROP=0
B: EV=11
B: MSC=8

root@IMX8MP-Tronlong:~#
```

图 37

执行如下命令，进行 DDR 读写测试。测试完成后，程序将会打印最终测试结果，包含读写平均传输耗时、读写平均传输速率、读写错误统计等信息。其中，写速率为 50.13MB/s，读速率为 50.11MB/s。

Target# ./dma_memcpy_demo -a 0xa4400000 -s 327680 -c 100 -d /dev/input/event1

```
root@IMX8MP-Tronlong:~# ./dma_memcpy_demo -a 0xa4400000 -s 327680 -c 100 -d /dev/input/event1
Open dma & input device node...
DMA: write: (time: 6121 us, rate: 51.08 MB/s, app time: 229 us, app rate: 1361.26 MB/s, write_error: 0)
read: (time: 6148 us, rate: 50.85 MB/s, app time: 1010 us, app rate: 309.29 MB/s, read_error: 0)

root@IMX8MP-Tronlong:~#
```

图 38

表 3

write/read	写操作	读操作
time	DMA 传输耗时，本次测试为 6121us	DMA 传输耗时，本次测试为 6148us
rate	DMA 传输速率，本次测试为 51.08MB/s	DMA 传输速率，本次测试为 50.85MB/s
app time	应用层写数据至驱动 Buffer 耗时，本次测试为 229us	应用层从驱动 Buffer 读取数据耗时，本次测试为 1010us
app rate	应用层写数据至驱动 Buffer 速率，本次测试为 1361.26MB/s	应用层从驱动 Buffer 读取数据速率，本次测试为 309.29MB/s
write_error/read_error	写数据过程中出错次数，本次测试为 0	读数据过程中出错次数，本次测试为 0

备注：0xa4400000 为预留的 DDR 地址空间，如下图所示，大小为 1024KByte。

```
23          reserved-memory {
24                  #address-cells = <2>;
25                  #size-cells = <2>;
26                  ranges;
27
28                  rpmsg_reserved: rpmsg@0x55800000 {
29                          no-map;
30                          reg = <0 0x55800000 0 0x800000>;
31                  };
32
33                  ddr_speed_reserved: ddr_speed@a4400000 {
34                          no-map;
35                          reg = <0 0xa4400000 0 0x00100000>;
36                  };
37          };
```

图 39 tlimx8mp-evm-reserved-memory.dts

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com